EngageNet

Progress Update - August 26, 2026

Lado Turmanidze, Luka Javakhishvili, Mariami Gadelia, Keso Chikhladze

Supervisors: Dr. Philipp Müller & Beso Mikaberidze

August 26, 2026

The Short Version

What happened since May

The model was built, it trained, and it produced numbers - but the numbers were bad.

I went looking for *why*, and found six problems. Two of them are not opinions: they are arithmetic. The model literally could not do what we designed it to do.

All six are now fixed in code. The paper has been rewritten to match. Nothing has been retrained yet - the GPUs are busy.

Today's Agenda

1. **Recap** - what the task is, what the model does
2. **Where we stood** - the numbers that started this
3. **The map** - a small theorem telling us where to look
4. **Problem 1** - the model had no memory
5. **Problem 2** - the modalities never talked to each other
6. **Problem 3** - we predicted the wrong thing
7. **Problem 4** - the loss function had three bugs
8. **Results & what is still open**

Part 0: What Is The Task?

The setup

Two people talk over a video link. One is the **expert** (explains something they love), one is the **novice** (listens, asks questions).

Human annotators watch the recording and move a slider to say *how engaged is this person, right now*.

Our job

For **every frame** of video, and for **each of the two people**, predict that slider value.

$y_t \in [0, 1]$, 25 frames per second

A 10-minute session is $\sim$15,000 numbers per person. We predict all of them.

Recap - The Data We Get

Four feature types $\mathcal{F}$

- eGeMAPS v2 - acoustic descriptors
- W2v-BERT 2.0 - learned speech embedding
- OpenFace2 - facial action units
- OpenPose - body keypoints

Two roles $\mathcal{R}$

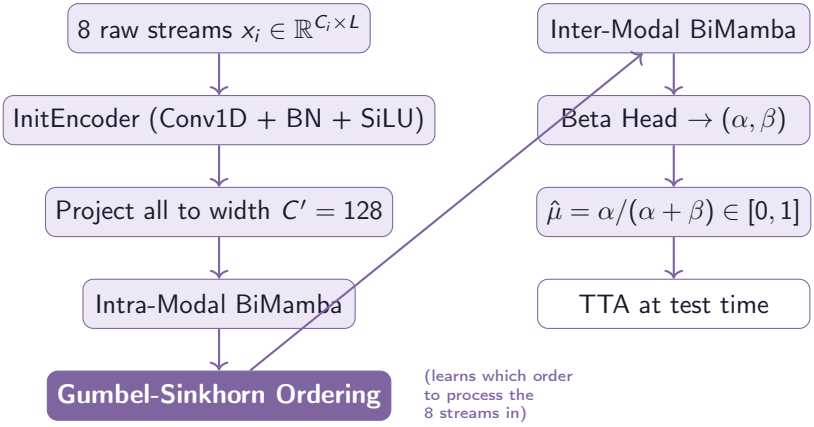
Separate camera and microphone per person, so **every feature type arrives twice**:

$$M = |\mathcal{F}| \times |\mathcal{R}| = 4 \times 2 = 8$$

8 parallel streams into the network.

Remember this $M = 8$. Half of today's talk is about what the model does with these 8 streams.

Recap - Architecture (as of May)



Recap - What A State-Space Model Actually Does

One line of maths, and it is the whole idea

$$\xi_t = \bar{A}\,\xi_{t-1} + \bar{B}_t u_t$$

ξ_t is the model's **memory** at step t. Each step it keeps a fraction $\bar{A}$ of what it remembered, and adds in the new input u_t.

Why this shape

$\bar{A} \in (0, 1)$, so old information fades smoothly instead of blowing up. Nice and stable.

Why it is fast

Looks sequential, but the update is *affine*, so we can compute all L steps with an associative scan in $O(\log L)$ depth instead of $O(L)$.

Keep $\bar{A}$ in mind. Problems 1 and 2 are both about this one number.

Recap - Why Beta Instead Of A Plain Number

The model outputs a distribution

Not one number, but two: (α, β), defining

$$p(y \mid x) = \mathrm{Beta}(\alpha, \beta), \quad y \in [0, 1]$$

Prediction is the mean $\hat{\mu} = \frac{\alpha}{\alpha+\beta}$.

Why bother

The *spread* tells us how sure the model is:

$$\mathrm{Var}[y] = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$$

Test-time adaptation uses this to pick which unlabelled frames are safe to learn from.

Beta lives on $[0, 1]$ - exactly the range of engagement. A Gaussian would put probability on -0.3 and 1.4, which are not things.

What training gave us

- Best validation CCC: **0.0227**
- Reached at epoch 2, then flat
- Correlation $\rho \approx 0.229$
- 7.2M parameters

What the leaderboard looks like

Official baseline	0.400
2025 winner	0.699
Us	**0.02**

This is not "needs tuning". This is "something is structurally wrong". So instead of tuning, I went looking for what.

Split CCC into two factors

Let $\varphi = \sigma_{\hat{y}}/\sigma_y$ (are the predictions as spread out as the truth?) and $\omega = (\mu_{\hat{y}} - \mu_y)/\sqrt{\sigma_{\hat{y}}\sigma_y}$ (are they centred right?). Then

$$\text{CCC} = \rho \cdot C_b, \qquad C_b = \frac{2}{\varphi + \varphi^{-1} + \omega^2}$$

Proposition

CCC $\leq \rho$, with equality only when $\sigma_{\hat{y}} = \sigma_y$ and $\mu_{\hat{y}} = \mu_y$.

Proof. $\varphi + \varphi^{-1} \geq 2$ for any $\varphi > 0$ (AM-GM), and $\omega^2 \geq 0$. So the denominator of C_b is at least 2, giving $C_b \leq 1$. □

Translation: correlation is a hard ceiling. Rescaling the output can never beat it.

What The Theorem Told Us

Our numbers

$$\rho = 0.229, \qquad \text{CCC} = 0.21$$

$$\implies C_b = \frac{0.21}{0.229} \approx 0.92$$

We were already **92% calibrated**.

So what does that mean

Fiddling with output scaling could buy us at most $0.229 - 0.21 = 0.02$.

Every remaining point has to come from ρ - from the model getting the *ordering* of frames right.

The model could not tell engaged moments from bored ones. Everything that follows is about *why not*.

Problem 1: The Model Had A Goldfish Memory

How long does an SSM remember?

Stop feeding it input. Then $\xi_{t_0+k} = \bar{A}^k \xi_{t_0}$: memory decays geometrically. The standard measure is how long until it drops to e^{-1}:

$$\bar{A}^{\tau_{\text{eff}}} = e^{-1} \implies \tau_{\text{eff}} = \frac{-1}{\ln \bar{A}} = \frac{1}{\Delta |A|}$$

Now plug in what we actually initialised

We set $A_{\log} = 0$ and $s_\Delta = 0$. That gives $|A| = e^0 = 1$ and $\Delta = \text{softplus}(0) = \ln 2 \approx 0.693$, so

$$\tau_{\text{eff}} = \frac{1}{0.693} \approx 1.44 \text{ frames} = 58 \text{ ms}$$

Problem 1 - Why 58 ms Is A Disaster

The mismatch

Engagement changes over **seconds**. Our model could see **58 milliseconds**.

That is about one and a half frames. It is like judging whether someone is enjoying a film by looking at a single blink.

And it gets worse

$A_{\log} = 0$ means *every one* of the $N = 16$ state coordinates got the **same** decay rate.

We paid for 16 memories at different timescales and received 16 identical copies of one very short memory.

Reference Mamba initialises Δ between 10^{-3} and 10^{-1}. Ours was 0.693 - **seven times larger than their maximum**.

Problem 1 - The Fix

S4D-Real initialisation

Give each state coordinate its own decay rate, spread geometrically:

$$A_{\log}[d, n] = \ln(n+1) \implies a_{d,n} = -(n+1), \quad n = 0, \ldots, N-1$$

and spread Δ log-uniformly over $[10^{-3}, 10^{-1}]$ by inverting the softplus, $s_\Delta = \ln(e^\Delta - 1)$.

Before

$|A|$: one value
τ_{eff}: 1.4 frames (58 ms)

After (measured)

$|A|$: 16 distinct values, $1 \to 16$
τ_{eff}: $10 \to 1000$ frames
(0.4 to 40 seconds)

Two lines of code. The model can now choose its own memory span per channel.

Problem 2: The Modalities Never Talked To Each Other

The whole point of the fusion stage

Take the 8 processed streams and let them share information - so the model can notice "the face says bored *but* the voice says excited".

What we found

We had **two different architectures**: one described in the paper, one actually running in the code.

They were different from each other, and both were broken - in different ways.

This is the embarrassing one. It is also the one with the biggest payoff.

Problem 2a - What The Paper Described

Lay the 8 streams end to end, scan along that line

Each stream is $C' = 128$ channels wide, so the line is $MC' = 1024$ positions long and **each modality occupies a block of 128**.

The telephone game problem

To get from one modality to the next, information must survive $C' = 128$ steps. Each step keeps a fraction $\bar{A}$:

$$\bar{A}^{C'} = 0.5^{128} \approx 3 \times 10^{-39}$$

The smallest number `float32` can represent is $\approx 1.2 \times 10^{-38}$.

The message does not arrive faint. It arrives as exactly zero.

Like whispering down a line of 128 people where each one halves the volume.

Problem 2b - What The Code Actually Did

Concatenate the modalities as *features*, scan along time

Different design. Here modalities *do* mix - through a big dense layer $W \in \mathbb{R}^{MC' \times MC'}$ that touches everything at once. So fusion happens.

But two things go wrong.

The ordering did nothing

Our whole Gumbel-Sinkhorn machinery learns a permutation P. But

$$W(P \otimes I_{C'})\, U = W'U, \quad W' := W(P \otimes I_{C'})$$

A dense layer can **absorb** any reordering into its own weights. The permutation was decorative.

It ate the whole model

$D = MC' = 1024$, and the block has five 1024×1024 matrices:

$$5.36\text{M of } 7.20\text{M} = 74\%$$

Three quarters of our parameters, on 31 training sessions.

Problem 2 - The Fix: Make Modality The Sequence

Stop unpacking modalities into channels

Treat each modality's whole C'-vector as **one position**. The sequence is then just the 8 modalities, and we run it once per time step:

$$H^{(t)} = \text{BiMamba}_{\text{modality}}\big(\tilde{u}_1^{(t)}, \ldots, \tilde{u}_M^{(t)}\big), \qquad t = 1, \ldots, L'$$

Distance between modalities

Was $C' = 128$ steps $\rightarrow$ now **1 step**.
Attenuation $\bar{A}^{128} \rightarrow \bar{A}$.

Parameter cost

Was $\Theta(M^2 C'^2) \rightarrow$ now $\Theta(C'^2)$.
5.36M $\rightarrow$ 96,640 (a 55x cut).

And now the learned ordering *matters*, because the scan really is sequential over modalities.

Problem 3: We Predicted The Wrong Thing

What we were doing

The challenge asks for **each person's** engagement. We were averaging the two into one number and predicting that:

$$\bar{y}_t = \tfrac{1}{2}\left(y_t^e + y_t^n\right)$$

Why averaging destroys signal

With both annotations having variance σ^2 and correlation r:

$$\mathrm{Var}(\bar{y}) = \tfrac{1}{4}\left[\sigma^2 + \sigma^2 + 2r\sigma^2\right] = \frac{\sigma^2(1+r)}{2}$$

If the two people's engagement moves *oppositely*, averaging flattens the curve towards a straight line.

Problem 3 - How Bad Is It? The Corpus Paper Measured It

Expert-novice correlation r

Language	r	Variance kept
German	-0.19	41%
English	-0.07	47%
French	-0.05	48%
Japanese	$+0.51$	76%

(Funk et al., 2024, Sec. 5.3.4)

Three of four are negative

On the European sessions we were throwing away **more than half** the signal before training even started.

And CCC divides by the target variance - so this is not recoverable later.

Japanese is the exception; the paper links it to a stronger tendency toward interpersonal harmony.

Problem 3 - The Fix (Easier Than Expected)

No new machinery needed

The two people were **never mixed on the way in** - their recordings enter as separate streams and stay separately indexed the whole way through.

So: attach **one output head per role** instead of one shared head. Each unimodal head is now trained against the person whose stream it reads.

Bonus 1

This is what the challenge actually asked for. We were solving a different problem than the one being scored.

Bonus 2

Gender is per person. With one averaged output, a session where the two differ had *no valid group label* - we were silently dropping those sessions from the fairness metric.

Problem 4: The Loss Function Had Three Bugs

The old loss, in full

$$\mathcal{L}_{\text{train}} = \mathcal{L}_{\text{NLL}}(\alpha, \beta, y) \; + \; \lambda_u \sum_{i=1}^{M} \mathcal{L}_{\text{NLL}}(\alpha_i, \beta_i, y), \qquad \lambda_u = 0.5$$

1. It rewarded being *calibrated*, but we are scored on being *correlated*
2. It paid most attention to the frames the model already knew
3. The 8 helper heads outweighed the main head by $4x$

Bonus defect: nothing in it ever asked the model to be fair.

Bug 4a - Calibrated Is Not The Same As Correlated

What the likelihood asks

"At *this* frame, is your predicted distribution right?"

One frame at a time. Never compares frames to each other.

What CCC asks

"Across *all* frames, do your predictions go up and down in the same pattern as the truth?"

And by Part 2, that is the only thing that can raise our score.

Fix: add the metric itself to the loss

$$\mathcal{L} \mathrel{+}= \lambda_c \left(1 - \widehat{\mathrm{CCC}}\right)$$

Every ingredient of CCC (two means, two variances, one covariance) is a smooth function of the predictions, so we can just differentiate it.

Bug 4b - The Loss Ignored The Hard Frames

Differentiate the Beta log-likelihood w.r.t. the mean

Writing $\mu = \alpha/(\alpha+\beta)$, $\kappa = \alpha+\beta$ (so κ = confidence), and using $\partial\alpha/\partial\mu = \kappa$, $\partial\beta/\partial\mu = -\kappa$, the two $\psi(\alpha+\beta)$ terms cancel and leave

$$\frac{\partial(-\ln p)}{\partial\mu} = \kappa\left[\ln\frac{1-y}{y} + \psi(\mu\kappa) - \psi((1-\mu)\kappa)\right] \;\propto\; \boldsymbol{\kappa}$$

Read that again

The correction is multiplied by the model's own confidence. Frames it is *already sure about* produce the biggest updates; uncertain frames produce almost none.

It is a teacher who only helps the students who already understand.

Bug 4b - The Fix, And Why It Needs stop_grad

Re-weight each frame

$$w_t = \text{stop\_grad}\left(\kappa_t^{-\beta_w}\right), \qquad \Longrightarrow \qquad \frac{\partial \mathcal{L}}{\partial \mu} \propto \kappa^{1-\beta_w}$$

$\beta_w = 0$ is the old behaviour, $\beta_w = 1$ removes confidence weighting entirely. We use $\beta_w = 0.5$.

Why the weight must be detached

If gradients could flow through w_t, the model would spot a shortcut: make κ huge, make w_t tiny, and the loss goes down *without predicting anything better*.

`stop_grad` means "compute this number, then treat it as a constant". It rebalances the update without becoming another thing to game.

Bug 4c - The Helper Heads Were Running The Show

The arithmetic

$M = 8$ helper heads, each weighted $\lambda_u = 0.5$:

$$8 \times 0.5 = 4$$

versus 1 for the main head.

And at test time we **only use the main head**.

The fix

Divide by M:

$$\frac{\lambda_u}{M} \sum_{i=1}^{M} \mathcal{L}_{\text{NLL}}(\alpha_i, \beta_i, y^{r(i)})$$

Now the helpers contribute 0.5 total, not 4, and adding more modalities does not silently change the balance.

80% of the gradient was coming from parts of the network we throw away at evaluation.

Bug 4d - We Measured Fairness But Never Asked For It

CDD is differentiable, so we can train against it

Split the batch into n_b quantile bins of the ground truth. Inside each bin, compare the two groups' average predictions:

$$\mathcal{L}_{\text{fair}} = \frac{\sum_{b=1}^{n_b} |\mathcal{B}_b| \left(\overline{\hat{\mu}}_b^{\,A} - \overline{\hat{\mu}}_b^{\,B}\right)^2}{\sum_{b=1}^{n_b} |\mathcal{B}_b|}$$

Squaring removes the sign, so it is smallest exactly when both groups get the same prediction at the same true engagement.

Honest caveat

This term is **zero** on a batch containing only one group. Gender is a session-level property, so most of our batches are single-group.

It may need window-level shuffling to do anything. Landing it last, and measuring first.

The New Loss, All Together

$$\mathcal{L} = \underbrace{\frac{1}{|\mathcal{R}|}\sum_{r}\mathcal{L}^{w}_{\mathrm{NLL}}(\alpha^{r},\beta^{r},y^{r})}_{\text{per-frame fit, rebalanced}} + \underbrace{\lambda_c(1-\widehat{\mathrm{CCC}})}_{\text{ordering}} + \underbrace{\frac{\lambda_u}{M}\sum_{i}\mathcal{L}^{w}_{\mathrm{NLL}}(\alpha_i,\beta_i,y^{r(i)})}_{\text{per-modality}} + \underbrace{\lambda_f\mathcal{L}_{\mathrm{fair}}}_{\text{fairness}}$$

Term	**Asks the model to...**	**Weight**
Beta NLL (reweighted)	get each frame's distribution right	1, $\beta_w = 0.5$
$1-\widehat{\mathrm{CCC}}$	rank frames correctly	$\lambda_c = 1.0$
Per-modality NLL	keep every stream individually useful	$\lambda_u/M = 0.0625$
Squared CDD	treat groups equally at equal truth	$\lambda_f = 0.05$

What Changed, In Numbers

	Before	**After**
Memory span τ_{eff}	1.4 frames (58 ms)	10–1000 frames (0.4–40 s)
Distinct decay rates	1	16
Cross-modal distance	$C' = 128$ steps	1 step
Fusion block parameters	5,360,000 (74%)	96,640 (5.0%)
Total parameters	7,197,898	2,064,844
Output heads	1 shared	2 (one per person)
Loss terms	2	4
Permutation actually used	no	yes

All verified on CPU. Smoke tests pass, forward pass is finite, gradients flow.

Status - Honest Version

Done ✓

- All six fixes in code
- Paper rewritten to match
- Structural checks pass
- Parameter counts confirmed

Not done

- **Nothing has been retrained**
- MICM GPUs occupied by another job (∼75 GB across both cards)
- So: no new CCC number to show you today

I can tell you the model can now do things it previously could not.
I cannot yet tell you how much that is worth.

Still Open

Blocking

- **MPIIGroupInteraction** - we do not have it. It is 1 of the 4 test sets, so we currently cap at 3/4 of the score
- GPU availability

Known gaps

- MC dropout (paper describes it, code has no dropout)
- Hungarian hard permutation - not implemented
- Score matrix missing $1/\sqrt{d_s}$ scaling
- TTA never run end-to-end

Next, in order

1. Retrain with $\eta = 5 \times 10^{-4}$ once a GPU frees up
2. Measure ρ, not just CCC
3. Ablate the four loss terms
4. Chase MPIIGI access

What I Would Like From You

Questions I cannot answer alone

- **MPIIGroupInteraction** - does anyone have a contact who can speed this up? Without it we lose a quarter of the score by default.
- **Training strategy** - one model on the union of corpora, or separate models per corpus? The challenge requires us to state which. I lean towards one model, but have not tested it.
- **Batch composition** - the CCC and fairness terms both want batches that span sessions. Reworking the loader is real effort. Worth it?

Questions?